

- 1) Scrivere una classe **Contatto** con le proprietà id (int), nome, cognome, email, telefono (tutte String) per rappresentare un contatto e una classe **Rubrica** che consenta di rappresentare i vostri contatti. La classe Rubrica conterrà al suo interno una ArrayList<Contatto> e metodi per aggiungere un contatto, eliminare un contatto di cui si indica l'id, restituire un contatto di cui si indica l'id, visualizzare tutti i contatti con un determinato cognome.  
Implementate i metodi per aggiunta, cancellazione e ricerche nella classe **Rubrica** e scrivete un main per testare le funzionalità della classe. I metodi potrebbero essere i seguenti:  

```
void aggiungi(Contatto c);  
void elimina(int id);  
Contatto cerca(id);  
void visualizza(String cognome);
```
- 2) Creare una classe **Libro** incapsulata, in modo tale che astragga il concetto di un libro che può essere venduto in una libreria. Tra i campi definiti dalla classe **Libro**, deve esserci il campo *genere* (inteso come genere letterario). Fare in modo che un libro possa essere associato solo ad un genere letterario compreso tra un insieme di generi letterari predefiniti, per esempio l'insieme costituito dai generi romanzo, thriller, saggio, manuale. Creare una classe che testi che gli oggetti **Libro** funzionino nella maniera corretta. (Suggerimento: utilizzate un array static e final di String per memorizzare i generi predefiniti).
- 3) Creare una classe **Point2D** per rappresentare punti geometrici in 2D. **Point2D** incapsula le coordinate x e y. Oltre ai soliti metodi per l'incapsulamento, prevedere un metodo *toString()* che descriva il punto come segue: "(<x>, <y>)".  
Create quindi una classe **Point3D** che estende la classe **Point2D** per incapsulare le coordinate x, y, z. Prevedere un metodo *toString()* che descriva un punto 3D come: "(<x>, <y>, <z>)".  
Scrivete infine un programma di esempio in cui create alcuni oggetti delle due classi e li visualizzate utilizzando i metodi *toString()*.

---

## ESERCIZI DEL LIBRO DI TESTO

### Esercizio 6.16 Classe incapsulata Rettangolo

Creare una classe Rettangolo con attributi privati base (double) e altezza (double). Implementare metodi getter e setter per gli attributi. Nei setter, assicurarsi che i valori passati siano positivi. Se un valore non è valido, stampare un messaggio di errore. Scrivere un costruttore che accetta base e altezza come parametri e inizializzare gli attributi usando this. Aggiungere un metodo calcolaPerimetro che restituisce il perimetro del rettangolo e un metodo calcolaArea che restituisce l'area del rettangolo.

### Esercizio 6.17 Classe Conto Bancario

Creare una classe ContoBancario con attributi privati saldo (double) e numeroConto(String). Implementare i metodi getter per entrambi gli attributi, e valutare se fornire metodi setter per numeroConto e saldo. Implementare i metodi pubblici deposita e preleva. Il metodo deposita deve aggiungere un importo al saldo, mentre preleva deve sottrarre un importo dal saldo solo se il saldo è sufficiente, altrimenti deve stampare un messaggio di errore. Creare due costruttori: uno con parametro numeroConto, e un altro con parametri numeroConto e saldo. Il saldo non può essere negativo. Creare infine una classe chiamata TestContoBancario che testa il corretto funzionamento dei costruttori, e dei metodi preleva e deposita in tutti gli scenari implementati.

Suggerimento: prima di scrivere il codice di TestContoBancario individuare tutti gli scenari da testare.

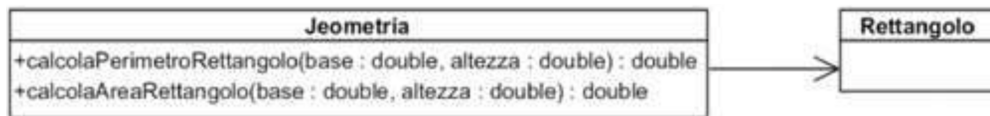
### Esercizio 6.25 Refactoring classe incapsulata Rettangolo

Partendo dalla soluzione dell'esercizio 6.16, rendere la classe Rettangolo una classe immutabile, senza modificare le sue caratteristiche. Dichiarare la classe all'interno di un package com.jeometria. Dichiarare la classe TestRettangolo all'interno del package com.jeometria.test e assicurarsi che continui a funzionare come prima.

Suggerimento: nel caso vogliate usare la riga di comando per questo e i prossimi progetti, ricordate che per compilare e lanciare una classe appartenente a un package bisogna usare l'intero fully-qualified name (cfr. paragrafo 2.4.2).

### Esercizio 6.26 Classe Jeometria

Partendo dalla soluzione dell'esercizio precedente, nel package com.jeometria creare una classe chiamata Jeometria che dichiara due metodi pubblici come riportati nel seguente diagramma delle classi:



Nella classe Rettangolo, non abbiamo riportato i dettagli perché superflui. Notiamo una freccia che va dalla classe Jeometria verso Rettangolo. Si tratta del simbolo di associazione, il quale implica che la classe Jeometria usa (e quindi dipende da) la classe Rettangolo (cfr. syntax reference 5.3). In Java, questo simbolo si traduce solitamente nella creazione di una variabile d'istanza della classe Rettangolo all'interno della classe Jeometria. Creare anche una classe TestJeometria nel package com.jeometria.test per testare la classe Jeometria.

### Esercizio 6.35 Refactoring Jeometria

La classe Jeometria dell'esercizio 6.26 contiene solo due metodi e nessuna variabile d'istanza. I due metodi non dipendono da variabili d'istanza, quale modificatore sembra opportuno utilizzare? Quindi modificare i due metodi con il modificatore scelto. Modificare di conseguenza anche la classe TestJeometria affinché sia più sintetica possibile.

### Esercizio 6.36 Jeometria Singleton

Creare un nuovo progetto partendo dal precedente, dove però la classe Jeometria viene trasformata in JeometriaSingleton che implementa il design pattern Singleton. Modificare di conseguenza anche la classe TestJeometria affinché sia più sintetica possibile.

### Progetto 6.2 JTombola (Versione 1.6)

Partendo dalla soluzione del progetto 5.2, incapsulare la classe Bussolotto nel modo ritenuto più corretto (decidere se inserire i metodi setter e getter) e inserirla nel package com.jtombola. Nello stesso package creare il tipo Tabellone tenendo presente che è formato da 6 schede da 15 numeri (non consecutivi) come mostrato nella figura seguente.

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|----|----|----|----|----|----|----|----|----|----|
| 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |
| 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 |
| 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 50 |
| 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60 |
| 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 |
| 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 80 |
| 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 90 |

Nella classe Tabellone aggiungere anche il metodo selezionaNumero che seleziona il numero passato come argomento (scegliere il modo più opportuno). La classe Tabellone deve essere una classe che implementa il pattern Singleton. Creare anche la classe Croupier che definisce un metodo estraiNumero che prima invoca il metodo dammiNumero della classe Bussolotto, e poi lo passa come argomento al metodo selezionaNumero della classe Tabellone. Creare una classe TestCroupier che testa il metodo estrai numero. Per verificare che tutto funzioni correttamente, inserire delle istruzioni di stampa all'interno solo della classe Croupier, come se spiegasse tutto quello che accade. Esempio di output:

Estraggo un numero dal bussolotto...

Estratto il numero 10.

Seleziono il numero 10 sul tabellone...

Numero 10 selezionato sul tabellone.

### Esercizio 7.18 Dati Immobiliare

Creare una applicazione chiamata "Immobiliare" che permetta di ricercare immobili. In particolare, si potranno vendere appartamenti, garage e palazzi. Creare una gerarchia sigillata di tipi che rappresentano i concetti prima espressi, all'interno di un package chiamato com.jimmobiliare.dat. Tenere conto che un palazzo è formato da una collezione di appartamenti. Inserire nelle varie classi i giusti attributi, metodi e costruttori, utilizzando i concetti imparati sino ad adesso (incapsulamento, super, this, toString, i modificatori corretti ecc.) Infine, creare un diagramma delle classi che rappresenta questi tipi con tutti i dettagli come documentazione.